

AI Multi-Paper Knowledge Synthesizer

🔗 "Why Technology X?" Interview Defense Guide

1. VECTOR DATABASE & EMBEDDING MODELS

Q1: Why Qdrant over Pinecone, Chroma, or FAISS?

Answer:

- **Native HNSW Index & Speed:** Written in Rust, Qdrant provides extremely low search latency with memory efficiency under heavy parallel queries.
- **Rich Metadata Payload Filtering:** Unlike basic FAISS, Qdrant allows filtering vectors by metadata (`paperId` , `authors` , `year`) during vector search without sacrificing speed.
- **Deployment Flexibility:** Supports seamless transition from zero-config in-memory embedded mode to production Qdrant Cloud without changing client code.

Q2: Why BAAI/bge-large-en-v1.5 over OpenAI (text-embedding-3) or Sentence-Transformers?

Answer:

- **Top MTEB Benchmark Performance:** BAAI bge-large-en-v1.5 is ranked among the highest-performing open retrieval embedding models on the MTEB benchmark.
- **1024-Dimensional Representation:** Captures rich, dense semantic nuances in dense scientific prose much better than 384-dim or 768-dim models.
- **Privacy & Cost Control:** Runs locally/self-hosted via Hugging Face Transformers, eliminating per-token API costs and latency overhead from proprietary endpoints.

2. LLM & INFERENCE ENGINE

Q3: Why Groq (Llama 3.3 70B) and Nvidia Nemotron 70B over OpenAI GPT-4o?

Answer:

- **Ultra-Low Latency (300+ tokens/sec):** Groq's custom LPU (Language Processing Unit) architecture streams syntheses in ~1.5 seconds, compared to 8-12 seconds on standard GPUs.
- **70B Parameter Open-Weights Quality:** Llama 3.3 70B matches GPT-4-grade reasoning on academic retrieval tasks at a fraction of the inference cost.
- **Resilience with Nemotron Fallback:** Nvidia Nemotron 70B serves as a secondary high-reasoning fallback engine for structured JSON comparison matrices.

3. BACKEND ARCHITECTURE & FRAMEWORKS

Q4: Why FastAPI (Python) + Express (Node.js) dual-backend architecture?

Answer:

- **Python FastAPI for AI/ML:** Native ecosystem integration with PyTorch, Hugging Face, sentence-transformers, and Qdrant Python SDK with async OpenAPI routing.
- **Node.js Express for App Hosting & SSR:** Handles non-blocking web server requests, Vite dev middleware, and fast I/O proxying.
- **Separation of Concerns:** Isolates heavy ML computations (vector math, chunking) from light web/session handling, making the backend modular and horizontally scalable.

4. SEARCH STRATEGY & DATA PIPELINE

Q5: Why Hybrid Search (BM25 + Cosine) instead of pure Vector Search?**Answer:**

- **Keyword Loss in Vector Search:** Dense embeddings understand concepts well ("machine learning"), but often miss specific terms ("transformer-xl", "loss = 0.04", paper titles).
- **BM25 Precision:** BM25 guarantees keyword matches for unique terms, acronyms, and citations.
- **Fused Score Balance:** Combining 0.65 Cosine + 0.35 BM25 delivers optimal precision and recall for scientific research.

Q6: Why 700-character Recursive Chunking with 100-character overlap?**Answer:**

- **Paragraph-Level Granularity:** Scientific paragraphs average 500-800 characters. 700 characters fits an entire thesis/finding without dilution.
- **Hierarchical Splitting:** Splitting by `[" ", " ", ". "]` preserves full sentences.
- **100-Char Overlap:** Prevents critical context (like variable definitions or reference links) from being severed at chunk boundaries.

5. FRONTEND STACK**Q7: Why React 19 + Vite 6 + Tailwind CSS v4 over Next.js?****Answer:**

- **Instant HMR & High-Performance UI:** Vite 6 + React 19 provides sub-millisecond hot module reloading and concurrent UI rendering for dynamic graph animations.
- **No Unnecessary SSR Complexity:** Single-page dashboard application doesn't require complex server component overhead; local client-side state manages paper selections cleanly.
- **Tailwind CSS v4 Engine:** High-performance utility styling with zero-runtime CSS bundle overhead.